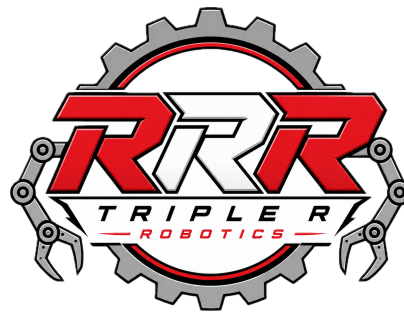
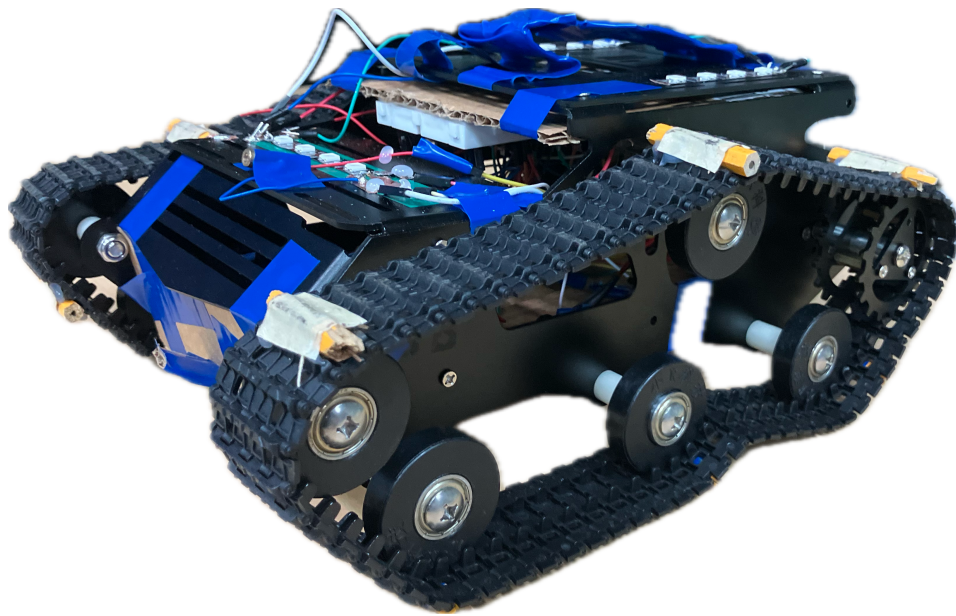


# Honors Principles of Engineering Final Project Report: **The Arduino Robot**

Author: Owen Reich  
Co-Engineers: Trent Rash, Will Robinson

Team Name: Triple R



May 8, 2026

## Abstract

We present a completed Arduino-powered tank robot equipped with numerous features and a central Bluetooth module for remote control via Microblue. The robot was designed to meet three categories: design of the robot, creativity of the robot's features, and speed in finishing an indoor obstacle course. The robot successfully completed the course in first place with a time of 5 minutes, 51 seconds. The key challenges were generating enough torque while turning and combating motor voltage sag.

# 1 Introduction

## 1.1 What is an Arduino Tank Robot?

An Arduino tank robot refers to a small robot with tank treads, controlled by an Arduino circuit board. An example of an Arduino tank robot is shown below. For a quick introduction to Arduino, see <https://shorturl.at/PoUd3>.

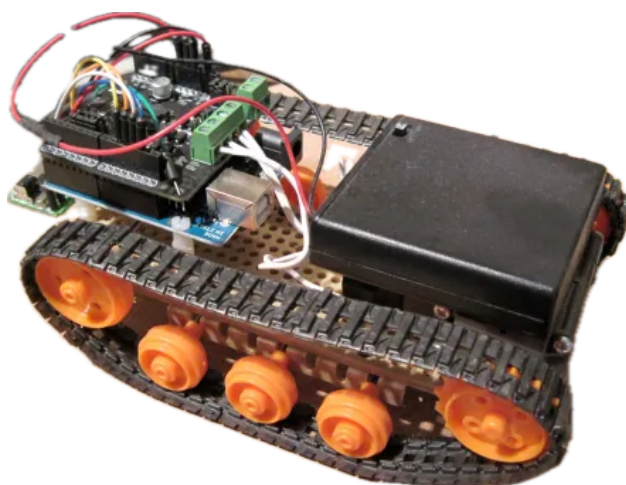


Figure 1: An example of an Arduino powered tank robot.

## 1.2 Why a Tank?

Robots with tank treads as opposed to wheels excel in gaining traction on rough or slippery ground due to their larger surface area and **grousers**. Grousers are ridges on tank treads that dig into the ground to gain traction. Treads also go over obstacles better than wheels. Finally, tank treads allow pivot turning, or turning in one place, which can be advantageous in small spaces.

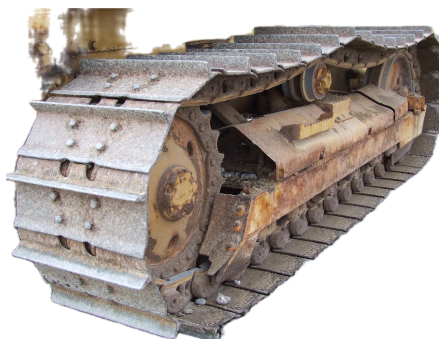


Figure 2: An example of grousers.

### 1.3 Goals of the Robot

Our goals were, in order of importance:

1. To be the fastest robot in order to win the race.
2. To utilize Bluetooth for remote control.
3. To contain all wiring inside the robot for durability.
4. To set ourselves apart from the other teams by implementing as many additional cosmetic features as possible.

We succeeded in 1, 2, and 4, and contained *almost all* wiring inside of the robot.

### 1.4 Important Learned Concepts

We learned:

- How to code basic Arduino C++.
- How to solder on small surfaces and components.
- How to use an Arduino's pins.
- How to create a battery voltage monitor.
- How to control Neopixel light strips with code.
- How to use PWM-capable pins to modulate pitch of a buzzer.
- How to connect to a phone with Bluetooth.

## 2 Components Used

### 2.1 Arduino Uno R3

We used an Arduino Uno R3 to control the circuit. For a quick introduction to Arduino, see <https://shorturl.at/PoUd3>.

### 2.2 Modules Used

We used the modules:

- HC-06 Bluetooth module.
- An active buzzer module (a passive buzzer works too).
- A 5v laser.
- 16 LED long Neopixel light strip.
- L293D motor controller.
- DC 9.6v motors.

as well as basic electronic components such as resistors, jumper wires, and batteries. The only component we were given and did not use was an ultrasonic distance sensor. We did not include it since the robot was to be remote controlled rather than autonomous.

## 2.3 Table of Materials and Components

| Component                  | Link                                                              | Purpose                                    |
|----------------------------|-------------------------------------------------------------------|--------------------------------------------|
| Arduino Uno R3             | <a href="https://shorturl.at/QLMZP">https://shorturl.at/QLMZP</a> | Controls the whole tank robot              |
| HC-06 Bluetooth Module     | <a href="https://shorturl.at/rbYA6">https://shorturl.at/rbYA6</a> | Connects to phone to command Arduino       |
| L293D Motor Controller     | <a href="https://shorturl.at/aKZa0">https://shorturl.at/aKZa0</a> | Powers motors with 9v but Arduino logic    |
| DC 9.6v Motors             | <a href="https://shorturl.at/T4X8R">https://shorturl.at/T4X8R</a> | Moves the whole robot                      |
| Ultrasonic Distance Sensor | <a href="https://shorturl.at/a14MG">https://shorturl.at/a14MG</a> | Unused                                     |
| Tank Chassis               | <a href="https://shorturl.at/ZHaI9">https://shorturl.at/ZHaI9</a> | Holds motors and circuitry in place        |
| 9v Batteries               | <a href="https://shorturl.at/UpEGa">https://shorturl.at/UpEGa</a> | Powers Arduino and motors                  |
| 5v Laser                   | <a href="https://shorturl.at/cef8t">https://shorturl.at/cef8t</a> | Fires a laser for fun                      |
| 5v Active Buzzer Module    | <a href="https://shorturl.at/YL3g8">https://shorturl.at/YL3g8</a> | Beeps when firing laser, low battery, etc. |
| Jumper Wires               | <a href="https://shorturl.at/FKbRI">https://shorturl.at/FKbRI</a> | Connects circuits and carries power        |
| Breadboard                 | <a href="https://shorturl.at/UqakL">https://shorturl.at/UqakL</a> | Central wiring hub for power + signals     |

## 3 Methodology and Schematics

### 3.1 Mechanical Assembly

Our team assembled the robot by first screwing each metal plate together and screwing the motors in place. Then, we assembled 12 idle wheels and two drive chain wheels, and put them in place with the track around them. Finally, we taped the Arduino to the ceiling of the tank, and attached the breadboard through double sided tape to the ceiling.

### 3.2 Circuit Assembly

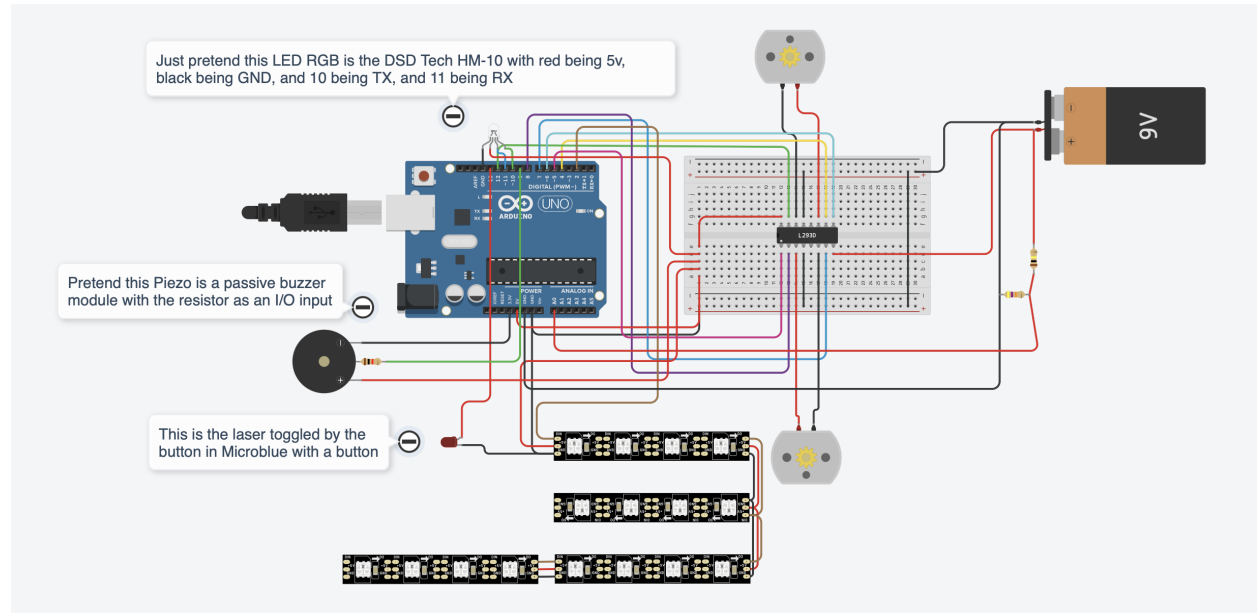


Figure 3: A TinkerCAD wiring diagram of the completed robot.

The figure above shows the completed wiring diagram for the robot. For detailed schematics, see [https://drive.google.com/file/d/1cCbtTfxRu\\_EoJ0xIGqpHG1L8UVJwb3Z/view?usp=sharing](https://drive.google.com/file/d/1cCbtTfxRu_EoJ0xIGqpHG1L8UVJwb3Z/view?usp=sharing). We developed



this circuit by starting with the very simple motor control circuit here:

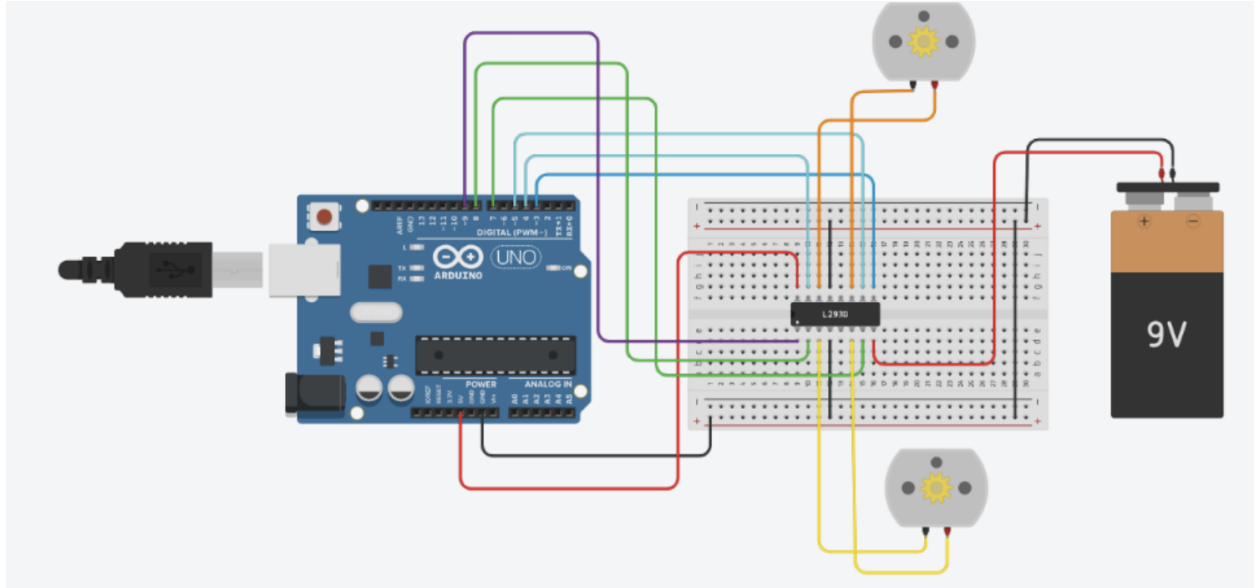


Figure 4: A basic motor control circuit with Arduino.

| Arduino (Input) Pin | Motor Controller (Output Pin) |
|---------------------|-------------------------------|
| 9                   | ENA Pin                       |
| 8                   | Input 1                       |
| 7                   | Input 2                       |
| 5                   | Input 3                       |
| 4                   | Input 4                       |
| 3                   | ENB                           |

Table 1: Diagram of simple Arduino to Motor Controller wiring.

From there, we added features in this order:

1. Bluetooth connection
2. Neopixel lights
3. Laser
4. Buzzer
5. Battery monitor

We did shift around pins for motors to make wiring more convenient. First, we shifted motor connections like in this table:

| Arduino (Input) Pin | Motor Controller (Output Pin) |
|---------------------|-------------------------------|
| 5                   | ENA                           |
| 8                   | Input 1                       |
| 7                   | Input 2                       |
| 4                   | Input 3                       |
| 12                  | Input 4                       |
| 6                   | ENB                           |

Table 2: The final Arduino to Motor Controller pinout diagram.

Then, we plugged in the Bluetooth module like so:

| Arduino (Input) Pin | HC-06 (Output) Pin |
|---------------------|--------------------|
| 11                  | RX Pin             |
| 10                  | TX Pin             |
| GND                 | GND                |
| 5v                  | VCC                |

Table 3: Arduino to Bluetooth module pinout.

One of the most common errors is a TX-RX flip, so if one encounters issues with the HC-06, flipping TX and RX is a harmless test that may fix problems. Secondly, we wired the Neopixel light strips, which have inputs of GND, VCC, and DIN (short for Data Input). To use the DIN pin, importing the Neopixel module into your Arduino sketch is essential:

```
#include <Adafruit_NeoPixel.h>
```

We used Pin 2 as our DIN pin. To connect the 5v laser, we simply hooked GND to GND and VCC to Pin 13. To connect the buzzer, we hooked GND to GND, VCC to the Arduino 5v, and I/O to Pin 9 for PWM functionality. Lastly, to add the battery voltage monitor, we added two resistors in this configuration to step down the voltage range of 6 – 9.6v to 2 – 3.1v, and ran the stepped down voltage into the A0 voltage sensor:

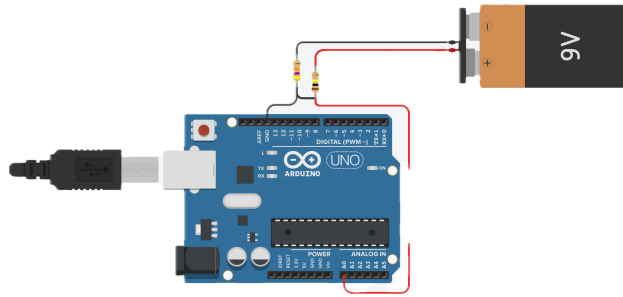


Figure 5: A model of a voltage divider for a battery voltage monitor with Arduino.

So the completed circuit becomes the one here: [https://drive.google.com/file/d/1cCbtTfxRu\\_EoJ0xIGqpHG1L8UVJwb3Z/view?usp=sharing](https://drive.google.com/file/d/1cCbtTfxRu_EoJ0xIGqpHG1L8UVJwb3Z/view?usp=sharing).

### 3.3 Power System Design

The motors and Arduino competed for limited battery current, especially during turns or acceleration. Under high load, the battery voltage sagged, reducing available voltage to the Arduino and auxiliary systems. Com-

petition mode disabled nonessential features such as lights and sound effects to reduce power consumption and improve system stability during the race.

### 3.4 System Architecture

The Arduino is the central controller of the tank. The HC-06 Bluetooth module is the communication layer that transmits phone signals to the Arduino. The L293D acts as the power driver to the motors. Neopixels function as a feedback system as well as cosmetic implements. The battery monitor system acts as a diagnostic subsystem that works with the Neopixels to provide data.

### 3.5 Programming

To program the robot, we started with the code given to us to control the motors, available here: <https://shorturl.at/N8uoZ>. Then, we switched the motor pins to accurately send signal.

Next, we worked on light code, and our team was the first to get the lights to work. This is our light code prototype code, which we sent to every other team: <https://shorturl.at/BpsC1>. Using the Neopixel package is crucial here as the correct signals are generated by the library.

To hook up the Bluetooth module, we collaborated with Walker's team, since Walker already had working Bluetooth code: <https://shorturl.at/3fLWq>. I modified this code then to actually send the joystick values to the motors by merging the motor control + light code with the Bluetooth code. Adding the laser was trivial then, as it is just one button in Microblue with one pin.

Finally, we programmed the buzzer by using PWM pin 9, and setting each note in the songs we wanted to play as a HZ value. Then, we used PWM and the buzzer's natural frequency to calibrate the note HZ values to PWM outputs experimentally. We also experimentally adjusted rhythms to make the melodies accurate. We programmed Seven Nation Army, Smoke on the Water, and Calon Lan, the tune for our Alma Mater.

Later, we implemented a "competition mode" and "play mode" using the on/off switch in Microblue:

| Play Mode                                                   | Competition Mode                    |
|-------------------------------------------------------------|-------------------------------------|
| Neopixel lights do various animations based on movement     | Neopixel lights are off             |
| Laser button shoots the laser, pulses the lights, and beeps | Laser button does nothing           |
| Buzzer plays when prompted                                  | Buzzer does not play                |
| Button b1 plays Calon Lan                                   | b1 does not play Calon Lan          |
| Comp → Play plays Smoke on the Water                        | Play → Comp plays Seven Nation Army |

Table 4: Comparison Of Competition Mode and Play Mode

### 3.6 Testing

To test each stage of features, we:

- Checked voltages throughout the circuit with a multimeter.
- Tested every input button to the motors with Microblue.
- Added temporary testing code to the script to output Bluetooth values through the serial monitor.

## 4 Creativity and Design

### 4.1 Hacking the Buzzer with PWM

A digital pin is the main pin on the Arduino, pins 0-13. Digital pins are either on, or off, giving out 5v or 0v. To work around this, people developed PWM. PWM stands for Pulse Width Modulation. The core idea

is that if you turn the digital pin on and off extremely fast, it will "average out" the voltage and the signal behaves similarly to an analog voltage when averaged over time.

An active buzzer module is a buzzer module with an oscillator unit inside that gives out one fixed pitch. Our module had three pins, being 5v, GND, and I/O. In order to make the buzzer give off different pitches, we used PWM to make the buzzer's oscillator underpowered so it would buzz slower, giving a lower pitch.

This essentially turned the active buzzer into a passive buzzer, which allowed us to play simple melodies and other sound effects.

## 4.2 Special Lighting and Laser Animations

Our lighting animations, discussed in 5.2.3 and 5.2.4, are one of the main differences between our team's robot and the others. Our lighting features

1. A pulsing idle animation.
2. A red flash for a sudden stop.
3. Green for forward, yellow for turning.
4. A quick flash for a laser fire.

Our lighting also features a *dynamic battery bar*, which measures voltage and prints the current battery to the first 4 neopixels. Our battery measurement system is accurate to 0.15v at every voltage between 6 and 9 volts<sup>1</sup>.

## 4.3 Competition and Play Modes

Our final adjustment to the robot was the addition of the play and competition modes, which separate essential and nonessential functions for maximal performance in competition. They are toggled with a switch in Microblue.

Our main insight here was that canceling all nonessential features would improve longevity and likely improve speed due to higher Arduino voltage. Note that competition mode does not affect the motor battery life.

# 5 Final Sketch Code Explanation

Our final sketch is available here: <https://shorturl.at/Aw4Lq>.

## 5.1 Main Setup Functions

As with any Arduino code, the sketch has two functions: `setup()` and `loop()`. Setup runs once as the Arduino gains power, and loop runs continually in the background. Although these imports occur before `setup()`, the sketch first imports the modules necessary for lighting, Bluetooth, and mathematical calculations.

### 5.1.1 Setting Output Pins, Data Structs, and Constants

First, the script sets all used digital pins as output pins, since none of them are used as inputs to read voltage (A0 is the only pin used for this purpose).

```
// ----- PIN DEFINITIONS -----  
#define LED_PIN      2  
#define NUMPIXELS    12  
#define LASER_PIN    13  
#define BUTTON_PIN   3
```

---

<sup>1</sup>When the motors are not running. When motors are running, voltage is still accurate but significantly lower due to voltage sag. This means that while the motors are running, the battery meter does not measure battery capacity accurately.

```

#define SPEAKER_PIN 9
#define BATTERY_PIN A0

// Motor Pins (L298N / L293D)
const int enA = 5, in1 = 8, in2 = 7; // Left Motors
const int enB = 6, in3 = 4, in4 = 12; // Right Motors

```

Next, the script sets up the fundamental data structure for the buzzer melody, being the note:

```

struct Note {
    uint16_t freq;
    uint16_t dur; // Duration in milliseconds
};

```

Third, the script sets crucial constants relating to how joystick data is interpreted, how battery voltage is calculated after being stepped down, how long to wait to check the battery, and the length of the laser pulse and Neopixel update rate:

```

// ----- CONSTANTS & CONFIG -----
const int rawCenter = 512;
const int rawDeadzone = 50;
const int motorDeadzone = 25;
const bool flipX = true;
const bool flipY = true;

const float R1 = 100000.0f, R2 = 47000.0f;
const float DIVIDER_RATIO = (R1 + R2) / R2;
const float BATTERY_FULL_V = 9.0f;
const float BATTERY_EMPTY_V = 6.4f;
const float BATTERY_PRESENT_V = 1.5f;

const unsigned long PIXEL_FRAME_MS = 30;
const unsigned long BATTERY_UPDATE_MS = 1000;
const unsigned long LASER_PULSE_MS = 180;
const unsigned long LASER_COOLDOWN_MS = 250;

```

Next, the script defines note frequencies according to the A= 440 hz tuning:

```

// ----- FREQUENCY MACROS -----
#define N_REST 0
#define N_G2 98
#define N_G3 196
#define N_A3 220
...
#define N_E5 659
#define N_F5 698
#define N_G5 784

```

Finally, the script programs the melodies for Seven Nation Army, Smoke on the Water, and Calon Lan. Shown is the Seven Nation Army program:

```

/**
 * Seven Nation Army (124 BPM)
 * Rhythm: Dotted-Quarter, Eighth, Quarter, Quarter, Quarter, Half, Half
 */
const Note PROGMEM SNA[] = {
    {N_E4, 550}, {N_REST, 50}, // Note 1 (Dotted Quarter)

```

```

...
{N_B3, 650}, {N_REST, 100} // Note 7 (Half)
};
const uint8_t SNA_LEN = sizeof(SNA) / sizeof(SNA[0]);

```

### 5.1.2 Setting States

At startup, the script initiates BTSerial on pins 10 and 11, which is how the Arduino translates Bluetooth signals into Serial (the language which Arduino communicates in). Then, the script sets laserActive, startupDone, competitionMode, and melodyPlaying all to false.

### 5.1.3 Light Animations

The setup function plays a small light animation to test if lights are working.

## 5.2 Main Loop Functions

### 5.2.1 handleBluetooth()

The handleBluetooth() function parses Bluetooth commands and decodes them into instructions for the laser, lights, competition mode, melodies, or motors. If a true signal is passed, the function handles the instructions through processCommand().

processCommand() is a list of different command parsers:

- The slider (code Sl0) sets maxbrightness to the input.
- Button 1 (code b0) uses the function fireLaserPulse().
- Button 2 (code b1) calls the function startMelody() for Calon Lan.
- The switch (code Sw0) calls the function setCompetitionMode().
- The D-pad or Joystick (both are code d0) activates driveArcade().

driveArcade() parses the two values sent by the D-pad or Joystick, being  $x$  and  $y$ .  $x \in [0, 1024]$  with 0 being fully left and 1024 being fully right, and  $x = 512$  means no lateral movement.  $y \in [0, 1024]$  with 0 being fully backward and 1024 being fully forward, with  $x = 512$  being no forward movement. The function then mathematically transforms the inputs into motor speeds between  $[-255, 255]$  with this:

```

int leftSpeed = constrain(currentY + currentX, -255, 255);
int rightSpeed = constrain(currentY - currentX, -255, 255);

applyMotor(enA, in1, in2, leftSpeed);
applyMotor(enB, in3, in4, rightSpeed);

```

### 5.2.2 updateBattery()

This function checks if the time since the battery was last checked is greater than the minimum battery check cooldown. If this is true, then the function then takes the input from A0 as  $i_b \in [0, 1024]$  and reverses it to get the raw voltage input from A0  $i_v \in [0v, 5v]$ . Finally, the function multiplies by the divider ratio induced from the  $100,000\Omega$  and  $47,000\Omega$  resistors, which is given by

$$V_{\text{out}} = V_{\text{in}} \frac{R_{\text{low}}}{R_{\text{high}} + R_{\text{low}}}.$$

Since  $R_{\text{low}} = 47k\Omega$  and  $R_{\text{high}} = 100k\Omega$ , we have:

$$V_{\text{out}} = V_{\text{in}} \frac{47k\Omega}{100k\Omega + 47k\Omega} = V_{\text{in}} \frac{47}{147}.$$



Therefore,  $V_{\text{in}} \approx 3.12765 \cdot V_{\text{out}}$ . Therefore, the actual voltage of the battery should be 3.127 times the read voltage of the A0 pin.

However, the function does not stop here, since one single voltage reading would often be inaccurate or fluctuate! The function uses EMA, or Exponential Moving Average to smooth the battery voltage readings. It sets:

$$\text{voltage reading} = 80\% \cdot \text{previous voltage reading} + 20\% \cdot \text{instant voltage reading}.$$

This means that the voltage reading behaves *similarly*<sup>2</sup> to the average of the previous 5 voltage readings. This is more efficient than a simple "average the last five readings" system however, since this version only remembers one value (previous voltage reading) instead of five.

### 5.2.3 updateLightingState()

The function first checks if the laser is currently being fired and pulses the Neopixels if so. This takes priority over other animations. Next, the function checks the current  $x$  and  $y$  movement values and sets lighting based on them. Finally, if no other action is taking place, the function uses this formula to make the lights "breathe" as an idle animation:

$$\text{brightness}(t) = \left( e^{\sin\left(\frac{t}{1000\text{ms} \cdot \pi}\right)} - \frac{1}{e} \right) \cdot 108.$$

The exponential term fluctuates between  $e^{-1}$  and  $e^1$  with period 2000ms. Combined with the  $-\frac{1}{e}$  term, the term inside the parentheses fluctuates between 0 and 2.3504. When scaled by 108, this becomes  $\text{brightness}(t) \in [0, 254]$ , which fits neatly into the 8-bit LED brightness level Neopixels take.

### 5.2.4 updatePixels()

updatePixels() handles Neopixel animations, specifically making every light smoothly transition into each state rather than flipping like a light switch. The main math here is:

```
curR[i] = (curR[i] * 2 + tarR[i]) / 3;
curG[i] = (curG[i] * 2 + tarG[i]) / 3;
curB[i] = (curB[i] * 2 + tarB[i]) / 3;
```

which sets:

$$\text{next color (r/g/b)} = \frac{2 \cdot \text{last color} + \text{target color}}{3}.$$

This makes the lights 1/3 closer to the target value every iteration. For example, if starting at (0,0,0) and the target is (255,255,255), then the values will be:

---

<sup>2</sup>The memory of the EMA is infinite, but past readings' weight decays exponentially. The EMA is well approximated by an average of the last 5 values.

| Frame Number | Current Value   |
|--------------|-----------------|
| 0            | (0, 0, 0)       |
| 1            | (85, 85, 85)    |
| 2            | (142, 142, 142) |
| 3            | (180, 180, 180) |
| 4            | (205, 205, 205) |
| 5            | (222, 222, 222) |
| 6            | (234, 234, 234) |
| 7            | (241, 241, 241) |
| 8            | (246, 246, 246) |
| 9            | (249, 249, 249) |
| 10           | (252, 252, 252) |
| 11           | (253, 253, 253) |
| 12           | (254, 254, 254) |
| 13           | (255, 255, 255) |

This smoothing curve has the exact equation  $x_n = T + (x_0 - T) \left(\frac{2}{3}\right)^n$ , where  $x_n$  is the  $n$ -th frame,  $x_0$  is the starting value, and  $T$  is the target value.

### 5.2.5 updateMelody()

updateMelody() first checks if a melody is currently playing or if playback is disabled. If both are false, then updateMelody() simply advances through the list of notes predefined before setup().

## 6 Results

### 6.1 Race Results

Our team finished first in the obstacle course, with a time of 5 minutes and 51 seconds. The main factors were:

- Better turning traction due to our added grousers.
- Better uphill traction due to added grousers.
- Faster straight line speed due to less voltage sag from being in Competition Mode, with no lights or extra features.
- Being able to go over obstacles rather than around them due to extra traction from grousers.
- Better course routing due to Will Robinson's continual guidance rather than the other team, whose driver was also the navigator.

### 6.2 Creativity Results

Our main creativity came from: hacking the buzzer to produce music; implementing two control modes; adding dynamic light animations rather than solid lighting; adding a battery power monitor, and visual battery bar.

## 6.3 Design Results

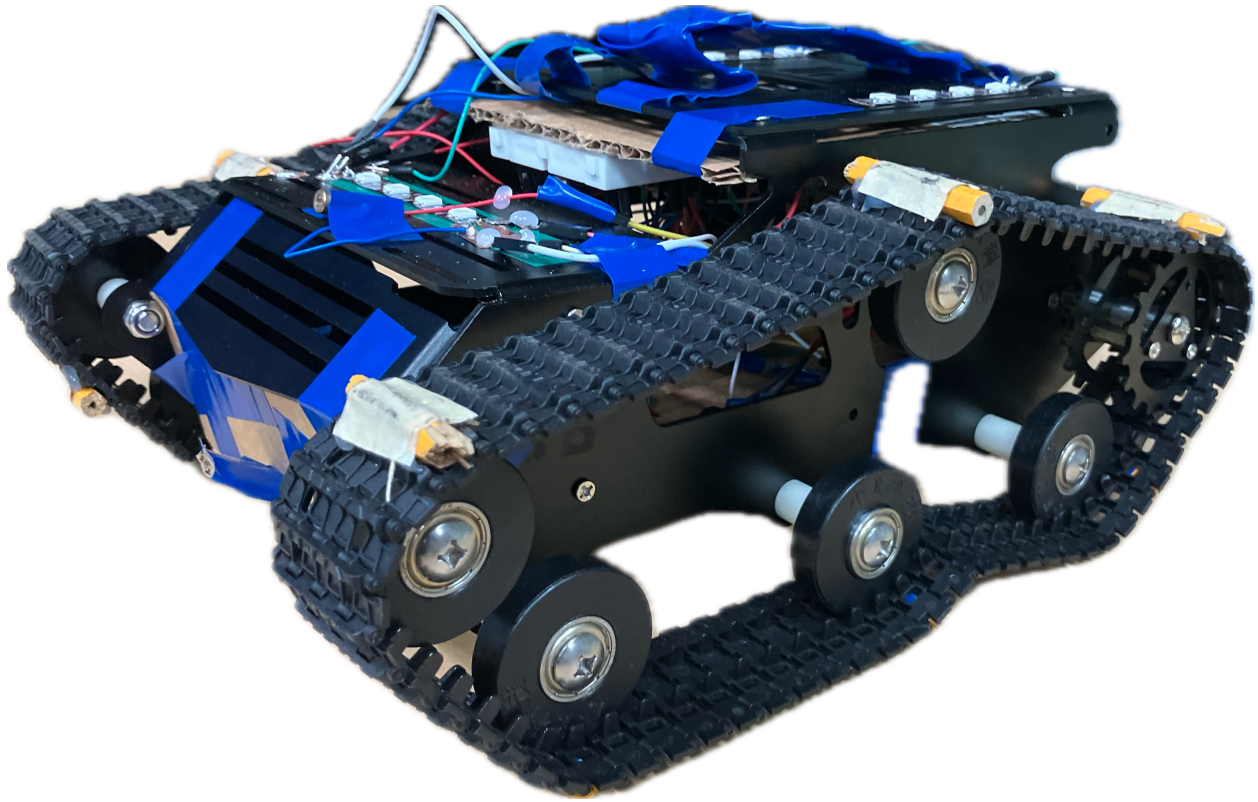


Figure 6: A photo of the finished robot a few minutes before competition.

Our design managed to keep a clean form factor, containing almost all wires within the metal shell except for external lighting wires. We also designed a cardboard hull to close the bottom so no wires could be pulled out or tangled.

We added a small handle built out of blue electrical tape on the top for easy carrying. The lights were all on the outside, with the frontmost strip being the battery monitor bar. The laser was positioned low and front facing, angled slightly up. The grousers were evenly spaced 10 tracks apart from each other, made of pencils, and attached with hot glue.

Finally, we included an openable hatch on the hull for replacing batteries, with a tab to keep it closed.

## 7 Conclusion

### 7.1 Goals Achieved

Overall, we succeeded in the main goal, to win the race on the obstacle course. Secondly, our robot probably had the most features, between the laser, lights, battery monitor, and musical buzzer. We mostly succeeded in keeping a clean form factor with wires inside the tank. Finally, we succeeded with implementing Bluetooth control as well as using almost every Microblue feature (d-pad, joystick, buttons, sliders, switches).

### 7.2 Technical Lessons

My main technical takeaway from this project was how to code Neopixel lighting, a buzzer, musical notes, mathematical animations, and a highly structured and interconnected sketch, all in CPP. Overall, I learned the most about programming safe, interconnected sketches, since almost every function had to have safety

features like "check if anything else competing is running." I learned a lot about top-down integrated system code architecture, and how to make sure the robot does not blow up due to memory or conflicting pin usages (we ran into MANY of these issues in testing).

### 7.3 Design Lessons

The biggest design lesson I took away from this is to always make a model in TinkerCAD or another circuit modeling software first! Our team was stuck for approximately a week trying to get the robot to work: the problem? A single wire mismatch.

Secondarily, we learned always to build circuits from the ground up. Start with the most essential feature; for us, it was motors. Then add the next thing, like Bluetooth. Then add all the other features. Instead of building the whole thing at once and hoping it works, separating the build process into stages helps diagnose errors and prevents a lot of wasted frustration.

### 7.4 Biggest Engineering Challenge

Our biggest engineering challenge was definitely wiring. We did not prioritize organized wiring, and we paid for it. Many problems were simply wires coming out or a single wire plugged into a wrong pin, which could have been avoided with cleaner wiring.

### 7.5 Future Improvements

The most important future improvements would be:

- Converting from breadboard to perf-board or a custom PCB.
- Replacing 9v batteries with LIPO battery packs.
- Adding a gearbox for extra torque or speed.
- Adding tensioners to the tracks for extra torque and efficiency.
- Adding an "autonomous mode" with an ultrasonic distance sensor.

## References

- [1] Adafruit Industries. Adafruit neopixel Überguide. <https://learn.adafruit.com/adafruit-neopixel-uberguide>, 2026. Accessed 2026-05-07.
- [2] Arduino. Arduino language reference. <https://docs.arduino.cc/language-reference/>, 2026. Accessed 2026-05-07.
- [3] Arduino. Arduino uno rev3. <https://docs.arduino.cc/hardware/uno-rev3/>, 2026. Accessed 2026-05-07.
- [4] Autodesk. Tinkercad circuits. <https://www.tinkercad.com/circuits>, 2026. Accessed 2026-05-07.
- [5] DSD Tech. Hc-06 bluetooth serial module documentation, 2026.
- [6] Microblue. Microblue app documentation, 2026.
- [7] Texas Instruments. L293, l293d quadruple half-h drivers datasheet. <https://www.ti.com/lit/ds/symlink/l293d.pdf>, 2026. Accessed 2026-05-07.